Assignment 6

Evelyn Barragan

02/18/2026

Question 1

1. import the random library.
2. Use `random.seed(10)` to initialize a pseudorandom number generator.
3. Create a list of 50 random integers from 0 to 15. Call this list `int_list`.
4. Print the 10th and 30th elements of the list.

You will need to use list comprehension to do this. The syntax for list comprehension is: <new_list> = `[<expression> for <item> in <iterable>]`. For this question your expression will be a randint generator from the random library and your iterable will be `range()`. Research the documentation on how to use both functions.

In [23]:
```
# your code here
# import random
import random
# initialize a pseudorandom number generator
random.seed(10)
# list of 50 integers from 0 to 15
int_list = [random.randint(0, 15) for n in range(50)]
print(int_list)
# print the 10th and 30th element of the list
print("10th element: ", int_list[9])
print("30th element: ", int_list[29])
```

```
[1, 13, 15, 0, 6, 14, 15, 8, 5, 1, 15, 10, 2, 7, 11, 1, 13, 4, 11, 12, 13,
9, 8, 14, 5, 9, 11, 4, 14, 7, 14, 12, 1, 0, 7, 4, 6, 9, 11, 7, 10, 14, 13, 1
5, 2, 10, 5, 7, 13, 7]
10th element:  1
30th element:  7
```

Question 2

1. import the string library.
2. Create the string `az_upper` using `string.ascii_uppercase`. This is a single string of uppercase letters

3. Create a list of each individual letter from the string. To do this you will need to iterate over the string and append each letter to the an empty list. Call this list `az_list`.

4. Print the list.

You will need to use a for-loop for this. The syntax for this for-loop should be:

```
for i in string>:    <list operation>
```

In [24]:
```
# your code here
# import string library
import string
# create string az_upper
az_upper = string.ascii_uppercase # single string of uppercase letters
# create a list for each individual letter from the string
az_list = [] # initialize list
for i in az_upper:
    az_list.append(i)
# print the list
print(az_list)
```

```
['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O',
'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z']
```

Question 3

1. Create a set from 1 to 5. Call this `set_1`.

2. Create a set from int_list. Call this `set_2`.

3. Create a set by finding the `symmetric_difference()` of `set_1` and `set_2`. Call this `set_3`.

4. What is the length of all three sets?

In [25]:
```
# your code here
# unordered collection with no duplicates
# mutable
# create a set from 1-5
set_1 = {1, 2, 3, 4, 5}
# create a set from int_list
set_2 = set(int_list)
# create a set by finding the symmetric_difference of set_1 and set_2
set_3 = set_1.symmetric_difference(set_2) # will return all the elements tha

print(set_1)
print(set_2)
print(set_3)

#length of all three sets
print("Length of set 1: ", len(set_1))
```

```
print("Length of set 2: ", len(set_2))
print("Length of set 3: ", len(set_3))
```

```
{1, 2, 3, 4, 5}
{0, 1, 2, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15}
{0, 3, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15}
Length of set 1:  5
Length of set 2:  15
Length of set 3:  12
```

Question 4

1. Import default dict and set the default value to 'Not Present'. Call this dict_1.
2. Add `int_list`, `set_2`, and `set_3` to `dict_1` using the object names as the key names.
3. Create a new dictionary, `dict_2`, using curly bracket notation with `set_1` and `az_list` as the keys and values.
4. Invoke the default value of `dict_1` by trying to access the key `az_list`. Create a new set named `set_4` from the value of `dict_1['az_list']`. What is the lenght of the difference between `dict_2['az_list']` and `set_4'?
5. Update `dict_2` with `dict_1`. Print the value of the key `az_list` from `dict_2`. What happened?

In [26]:
```
# your code here
# import default dict
from collections import defaultdict
# set default value
def def_value():
    return "Not Present."
dict_1 = defaultdict(def_value)
# add int_list, set_2, set_3, to dict_1 using the object names as the key na
dict_1["int_list"] = int_list
dict_1["set_2"] = set_2
dict_1["set_3"] = set_3

# create a new dictionary, using curly bracket notation with set_1 and az_li
dict_2 = {"set_1":set_1, "az_list":az_list}

# invoke the default value of dict_1 by trying to acccess key az_list
print(dict_1["az_list"])

# create new set, set_4 from the value of dict_1['az_list']
set_4 = set(dict_1["az_list"])

# length difference between dict_2["az_list"] and set_4
print("The length difference between dict_2['az_list'] and set_4 is: ", len(

# update dict_2 with dict_1
```

```
dict_2.update(dict_1)

# print value of the key az_list from dict_2
print(dict_2["az_list"])
```

```
Not Present.
The length difference between dict_2['az_list'] and set_4 is:  16
Not Present.
```

When dict_2.update(dict_1) is executed, the keys from dict_1 are merged into dict_2. Since both dictionaries contain the key "az_list", the value from dict_1 overwrites the original value in dict_2. This happens because update() replaces existing keys with new values if the keys match.

Question 5

A palindrome is a word, phrase, or sequence that is the same spelled forward as it is backwards. Write a function using a for-loop to determine if a string is a palindrome. Your function should only have one argument.

In [27]:
```
def palindrome(string):
    string = string.lower()
    backward_string = ""
    for i in range(len(string) - 1, -1, -1):
        backward_string += string[i]

    if string == backward_string:
        return True
    else:
        return False

palindrome("Racecar")
```

Out[27]:
```
True
```

In [28]:
```
palindrome("HelloWorld")
```

Out[28]:
```
False
```

Question 6

Two Sum - Write a function named two_sum() Given a vector of integers nums and an integer target, return indices of the two numbers such that they add up to target. You may assume that each input would have exactly one solution, and you may not use the same element twice. You can return the answer in any order.Use defaultdict and hash maps/tables to complete this problem.

Example 1: Input: nums = [2,7,11,15], target = 9 Output: [0,1] Explanation: Because nums[0] + nums[1] == 9, we return [0, 1].

Example 2: Input: nums = [3,2,4], target = 6 Output: [1,2]

Example 3: Input: nums = [3,3], target = 6 Output: [0,1]

Constraints: 2 <= nums.length <= 104 –109 <= nums[i] <= 109 –109 <= target <= 109
Only one valid answer exists.

In [29]:
```
def two_sum(nums, target):
    store = defaultdict(lambda: None)
    for i,value in enumerate(nums):
        j = target - value
        if j in store:
            return [store.get(j), i]
        else:
            store[value] = i

two_sum([2, 7, 11, 15], 9)
```

Out[29]:
```
[0, 1]
```

In [30]:
```
two_sum([3, 2, 4], 6)
```

Out[30]:
```
[1, 2]
```

In [31]:
```
two_sum([3, 3], 6)
```

Out[31]:
```
[0, 1]
```

In [ ]:
```

```